

Driving the PWM Servo

Experiment Objective

This section will guide you through driving a PWM servo with the MicroROS-V2 control board. Through this tutorial, you will learn how to use the ESP32-S3's MCPWM (Motor Control PWM) peripheral to generate a 50Hz servo control signal, set the rotation angle of the servo, and configure independent angle range limits for different servos.

Experiment Principle

1. Keywords

Keyword	Description
Servo	A servo motor whose angle is controlled by a PWM signal; standard servos use a 50Hz frequency, with a pulse width of 500~2500us corresponding to -90°~90°
MCPWM	The ESP32-S3's motor control PWM peripheral, which can generate multiple independent PWM signals and supports timer, comparator, and generator configuration
Timebase Resolution	The counting frequency of the MCPWM timer; at 1MHz resolution, each tick is 1us, which determines the PWM precision
PWM Period	The duration of a complete PWM signal; the standard period for a servo is 20ms (corresponding to a 50Hz frequency)
Pulse Width	The duration that the PWM signal stays high; a servo determines its rotation angle by changing the pulse width
Duty Cycle	The ratio of the high-level time to the period; changing the duty cycle changes the servo's output angle

2. Technical Architecture

The program uses the ESP32-S3's MCPWM peripheral to generate two 50Hz servo control signals, driving the PWM servos on the S1 (GPIO8) and S2 (GPIO16) interfaces respectively. The MCPWM timer works at 1MHz resolution with a period of 20000 ticks (20ms). The angle-to-pulse-width conversion formula converts the target angle (-90°~90°) into a comparator value, changing the PWM pulse width to control the servo to rotate to the specified angle. S1 and S2 each have independently configured angle limit ranges.

Main component files:

- `main/main.c` — main program entry, controls the servo swing demonstration
- `components/servo/servo.c` / `servo.h` — servo driver implementation and interface declarations

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB data cable	✓ Required (for power supply and serial monitor)
SG90 or similar PWM servo	✓ Required (for testing the S1 interface)

2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Connect the servo signal wire to the S1 (GPIO8) or S2 (GPIO16) interface of the control board

Servo wiring description

The servo interfaces on the MicroROS-V2 control board are defined as follows:

Servo interface	GPIO pin	Function
S1	GPIO8	Gimbal pitch servo (default angle 0°, range [-90, 90])
S2	GPIO16	Gimbal yaw servo (default angle -45°, range [-90, 20])

The standard servo has three wires: **brown (GND)**, **red (VCC 5V)**, and **orange (signal)**. The pin order is marked on the control board's servo interface; just plug it in directly.

3. Compile and Flash

```
cd ~/esp/base_samples/pwm_servo
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

After the program starts, servos S1 and S2 first return to their default angles. Then S1 swings back and forth between 90 degrees and -90 degrees, and S2 swings back and forth between -90 degrees and 20 degrees. Each angle is held for 1 second, and after 5 cycles the program stops.

4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

Experiment Result

After correctly flashing and running the program, the serial monitor outputs the following logs:

```
hello yahboom
I (xxx) MAIN: Nice to meet you!
I (xxx) MAIN: Servo_Set-> S1_Angle:0, S2_Angle:-45
I (xxx) MAIN: Servo_Set-> S1_Angle:90, S2_Angle:-90
I (xxx) MAIN: Servo_Set-> S1_Angle:-90, S2_Angle:20
I (xxx) MAIN: Circle Counter: 1
I (xxx) MAIN: Servo_Set-> S1_Angle:90, S2_Angle:-90
...
I (xxx) MAIN: Circle Counter: 5
I (xxx) MAIN: Servo_Set-> S1_Angle:0, S2_Angle:-45
I (xxx) MAIN: Circle End, exiting...
```

At the same time, the servos connected to the S1 and S2 interfaces swing back and forth according to the steps described in the logs.

Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/pwm_servo/main/main.c`
- Servo driver: `~/esp/base_samples/pwm_servo/components/servo/servo.h`, `servo.c`

1. Project Structure

```
pwm_servo/
├── CMakeLists.txt           # Top-level CMake project file
├── main/
│   ├── CMakeLists.txt      # main component build configuration
│   └── main.c              # Main program entry
└── components/
    ├── servo/
    │   ├── CMakeLists.txt   # Component configuration (depends on driver)
    │   ├── servo.c          # Servo driver implementation
    │   └── servo.h          # Servo driver interface declarations
```

2. Main Program Logic

The execution flow of `app_main` in `main.c`:

1. Print the startup information
2. Call `Servo_Init()` to initialize the two servos
3. After a 1-second delay, set S1 to 0° and S2 to -45° (initial position)
4. Enter a loop that executes 5 rounds of swinging:
 - S1=90°, S2=-90°, hold for 1 second
 - S1=-90°, S2=20°, hold for 1 second
5. After completion, return to the default angles and the program stops moving

3. Key Components

Servo Driver (servo.h / servo.c)

Pin and PWM parameter definitions:

Parameter definition file: `~/esp/base_samples/pwm_servo/components/servo/servo.h`

```
#define SERVO_GPIO_S1      8      // S1 servo signal line GPIO
#define SERVO_GPIO_S2     16     // S2 servo signal line GPIO
#define SERVO_TIMER_GROUP_ID 1    // MCPWM timer group (0 or 1)
#define SERVO_TIMEBASE_RESOLUTION_HZ 1000000 // Timebase resolution 1MHz, 1 tick = 1us
#define SERVO_TIMEBASE_PERIOD 20000 // PWM period 20000 ticks = 20ms (50Hz)
```

Servo control parameters:

```
#define SERVO_MIN_PULSEWIDTH_US 500 // Minimum pulse width 500us (corresponds to -90°)
#define SERVO_MAX_PULSEWIDTH_US 2500 // Maximum pulse width 2500us (corresponds to 90°)
#define SERVO_MIN_HD_ANGLE -90 // Minimum angle
#define SERVO_MAX_HD_ANGLE 90 // Maximum angle
```

Independent limit ranges for each servo:

```
// Servo S1: default angle 0°, range [-90, 90]°
#define SERVO_S1_DEF_ANGLE 0
#define SERVO_S1_MIN_ANGLE (-90)
#define SERVO_S1_MAX_ANGLE 90

// Servo S2: default angle -45°, range [-90, 20]° (-45° is directly ahead of the camera)
#define SERVO_S2_DEF_ANGLE (-45)
#define SERVO_S2_MIN_ANGLE (-90)
#define SERVO_S2_MAX_ANGLE 20
```

Servo ID enumeration:

```
typedef enum _servo_id {
    SERVO_ID_S1 = 0, // Servo S1
    SERVO_ID_S2 = 1, // Servo S2
    SERVO_ID_MAX
} servo_id_t;
```

Angle-to-pulse-width calculation (Servo_S1_Angle_To_Compare / Servo_S2_Angle_To_Compare):

The conversion relationship between the angle value and the pulse width:

$$\text{Pulse width (us)} = (\text{angle} - \text{minimum angle}) \times (\text{maximum pulse width} - \text{minimum pulse width}) / (\text{maximum angle} - \text{minimum angle}) + \text{minimum pulse width}$$

Substituting the specific values:

$$\begin{aligned}\text{Pulse width} &= (\text{angle} - (-90)) \times (2500 - 500) / (90 - (-90)) + 500 \\ &= (\text{angle} + 90) \times 2000 / 180 + 500\end{aligned}$$

For example, when the angle is 0°, the corresponding pulse width = $(0 + 90) \times 2000 / 180 + 500 = 1500\mu\text{s}$.

MCPWM initialization (`Servo_S1_Init` / `Servo_S2_Init`):

Taking S1 as an example, the MCPWM configuration flow is as follows:

1. Create the MCPWM timer:

```
mcpwm_timer_config_t timer_config = {
    .group_id = SERVO_TIMER_GROUP_ID,           // Timer group ID
    .clk_src = MCPWM_TIMER_CLK_SRC_DEFAULT,     // Default clock source
    .resolution_hz = SERVO_TIMEBASE_RESOLUTION_HZ, // 1MHz resolution
    .period_ticks = SERVO_TIMEBASE_PERIOD,      // 20000 ticks = 20ms
    .count_mode = MCPWM_TIMER_COUNT_MODE_UP,    // Up-counting mode
};
```

2. Create the operator and bind the timer:

- `mcpwm_new_operator` + `mcpwm_operator_connect_timer`

3. Create the comparator:

```
mcpwm_comparator_config_t comparator_config = {
    .flags.update_cmp_on_tez = true, // Update the compare value when the timer
    resets to zero
};
```

4. Create the PWM generator and bind the GPIO:

```
mcpwm_generator_config_t generator_config = {
    .gen_gpio_num = SERVO_GPIO_S1,
};
```

5. Set the PWM waveform actions:

- Output a high level when the timer resets to zero (`MCPWM_GEN_ACTION_HIGH`)
- Output a low level when the comparator matches (`MCPWM_GEN_ACTION_LOW`)

6. Enable and start the timer:

- `mcpwm_timer_enable` + `mcpwm_timer_start_stop`

Setting the servo angle (`Servo_Set_Angle`):

```
void Servo_Set_Angle(servo_id_t servo_id, int8_t angle)
```

- Select the corresponding servo according to `servo_id`
- The angle of S1 is inverted (`angle_servo = -angle`), because S1's mounting direction is opposite to the coordinate system definition

- Call `mcpwm_comparator_set_compare_value` to update the compare value and change the pulse width

API functions:

Function	Description
<code>void Servo_Init(void)</code>	Initialize the two servos (S1 and S2)
<code>void Servo_Set_Angle(servo_id_t servo, int8_t angle)</code>	Set the servo angle, unit: degrees

Common Problems

Problem	Cause	Solution
The servo does not move or jitters	Insufficient servo power supply	Make sure the servo uses an external 5V power supply; do not rely on USB power alone to drive high-torque servos
The servo gets stuck at its limit position	The angle setting exceeds the physical range	Check whether the angle value is within [-90, 90]; different servos may have different physical limits
The servo direction is opposite to expected	Angle conversion sign issue	The angle of S1 is already inverted in the code (<code>-angle</code>), which is designed according to the mounting direction